SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

Assessment Tool Weightage: 20%

Duration : 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Reverse Engineering

Code of Conduct:

I **NARAPPAGARI SRAVYA(192472192)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ANNEXURE A

REVERSE ENGINEERING

1. **Reverse Engineering a Recommendation System**

   A large-scale e-commerce platform provides highly accurate product recommendations using real-time data. Analyze the system and reverse engineer its architecture using PySpark and RDD operations. Identify how data is collected, processed, and transformed, and infer the role of intelligent agents in updating recommendations dynamically.

2. **Reverse Engineering a Smart Traffic Management System**

   A city-wide traffic system adapts signals in real time based on vehicle density and congestion patterns. Reverse engineer the underlying AI system by describing the data pipeline using RDD-based streaming, identifying agent interactions, and explaining how decisions are made under time constraints.

3. **Reverse Engineering a Fraud Detection Pipeline**

   A banking system flags fraudulent transactions within milliseconds. By observing its outputs and behavior, reconstruct the distributed AI pipeline using PySpark RDDs. Explain how feature extraction, anomaly detection, and agent-based alert mechanisms are likely implemented.

4. **Reverse Engineering a Personalized Learning System**

   An online learning platform adapts content difficulty based on student performance and engagement. Reverse engineer the system design by identifying how learner data is processed using RDD transformations, and explain how intelligent agents contribute to adaptive learning and feedback mechanisms.

5. **Reverse Engineering a Cloud-Based AI Monitoring System**

   A cloud platform automatically scales resources and detects system anomalies in real time. Reverse engineer the architecture by analyzing how PySpark processes large-scale logs, how RDD operations support fault tolerance, and how intelligent agents coordinate monitoring, scaling, and anomaly response.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

ANNEXURE A – REVERSE ENGINEERING

1. Reverse Engineering a Recommendation System

Objective

To analyze how a large-scale e-commerce platform provides accurate product recommendations in real time and reconstruct its possible architecture.

Inferred System Architecture

User Activities + Product Data + Purchase History

↓

Data Collection Layer

↓

PySpark RDDs

↓

Data Cleaning and Transformation

↓

Content-Based + Collaborative Filtering

↓

Recommendation Generation

↓

Intelligent Agents

↓

Dynamic Product Recommendations

↓

User Feedback Loop

1. Data Collection

The system likely collects:

- Products viewed by users
- Search history
- Items added to the cart

- Purchase history
- Product ratings and reviews
- User clicks and browsing behavior

2. RDD Data Processing

PySpark RDD operations can process millions of interactions across multiple machines.

- map() – transforms user and product data
- filter() – removes irrelevant records
- reduceByKey() – calculates product popularity
- join() – combines user behavior with product information
- groupByKey() – groups interactions by user

3. Recommendation Generation

The system may use:

Content-Based Filtering

- Recommends products similar to previously viewed or purchased products.

Collaborative Filtering

- Recommends products based on the behavior of similar users.

4. Role of Intelligent Agents

Intelligent agents likely:

- Monitor user behavior continuously
- Detect changes in user interests
- Update recommendations dynamically
- Analyze clicks and purchases
- Learn from user feedback

Conclusion

The reverse-engineered system uses **distributed PySpark processing, recommendation algorithms, and intelligent agents** to generate accurate real-time product recommendations.

2. Reverse Engineering a Smart Traffic Management System

Objective

To reconstruct the possible AI architecture of a city-wide traffic system that changes traffic signals based on real-time conditions.

Inferred Architecture

Traffic Cameras + Road Sensors + GPS Data

↓

Data Collection

↓

PySpark RDD Pipeline

↓

Data Processing and Analysis

↓

Traffic Density Detection

↓

Intelligent Agents

↓

Signal Control Decisions

↓

Traffic Signal Updates

1. Data Collection

The system likely receives data from:

- Traffic cameras
- Road sensors
- GPS devices
- Vehicle density sensors
- Traffic signal information

2. RDD-Based Processing

The incoming traffic data can be processed using:

- map() – processes each traffic record
- filter() – identifies congested roads
- groupByKey() – groups vehicles by location
- reduceByKey() – calculates vehicle density

3. Intelligent Agents

Different agents may perform different functions:

- **Monitoring Agent** – observes traffic conditions
- **Analysis Agent** – detects congestion
- **Signal Agent** – controls traffic signals
- **Coordination Agent** – coordinates nearby intersections

Decision Making Under Time Constraints

The system must make decisions quickly:

Traffic Data

↓

Detect Congestion

↓

Analyze Vehicle Density

↓

Agent Decision

↓

Change Signal Timing

For example, if one road has heavy congestion, the Signal Agent may increase its green-light duration.

Conclusion

The reconstructed system uses **distributed real-time data processing and multiple intelligent agents** to make fast traffic-control decisions.

3. Reverse Engineering a Fraud Detection Pipeline

Objective

To reconstruct how a banking system may detect fraudulent transactions within milliseconds.

Inferred Architecture

Financial Transactions

↓

Data Ingestion

↓

PySpark RDDs

↓

Data Cleaning & Processing

↓

Feature Extraction

↓

Anomaly Detection

↓

Suspicious Transactions

↓

Intelligent Agents

↓

Alert / Block / Review

↓

Feedback Loop

1. Feature Extraction

The system likely extracts features such as:

- Transaction amount
- Transaction time
- Location

- Device used
- Frequency of transactions
- Previous customer behavior

2. RDD Operations

Possible operations include:

- map() – extracts transaction features
- filter() – selects unusual transactions
- reduceByKey() – summarizes transactions for each account
- groupByKey() – groups transactions by customer

3. Anomaly Detection

The system compares current transactions with normal user behavior.

Examples of suspicious activity:

- Very large transaction amount
- Transactions from unusual locations
- Multiple transactions within seconds
- Unusual spending patterns

4. Intelligent Agent-Based Alerts

The system may include:

Detection Agent

- Identifies suspicious transactions.

Risk Assessment Agent

- Calculates the risk level.

Alert Agent

- Sends alerts to the bank or customer.

Learning Agent

- Uses confirmed fraud cases to improve detection.

Decision Process

Transaction

↓

Feature Extraction

↓

Anomaly Detection

↓

Risk Analysis

↓

Suspicious?

↙ ↘

Yes No

↓ ↓

Alert Approve

Conclusion

The reverse-engineered fraud detection pipeline combines **distributed processing, anomaly detection, and intelligent agents** for fast fraud identification and response.

4. Reverse Engineering a Personalized Learning System

Objective

To analyze how an online learning platform automatically changes learning content based on student performance and engagement.

Inferred Architecture

Student Activities + Quiz Scores + Engagement Data

↓

Data Collection

↓

PySpark RDDs

↓

Data Transformation & Analysis

↓

Student Performance Analysis

↓

Intelligent Agents

↓

Adaptive Content Selection

↓

Personalized Learning

↓

Feedback Loop

1. Learner Data Collection

The system likely collects:

- Quiz scores
- Assignment marks
- Time spent learning
- Videos watched
- Number of attempts
- Attendance
- Student engagement

2. RDD Transformations

PySpark can process data from thousands of students.

- map() – processes student records
- filter() – identifies weak areas
- reduceByKey() – calculates average scores
- groupByKey() – groups students by performance

3. Intelligent Agents

Performance Agent

- Analyzes student performance.

Content Agent

- Selects suitable learning materials.

Feedback Agent

- Provides immediate feedback.

Adaptive Agent

- Changes content difficulty based on student progress.

Adaptive Learning Process

Student Takes Quiz

↓

Analyze Performance

↓

Identify Weak Topics

↓

Intelligent Agent Decision

↓

Recommend Suitable Content

↓

Student Practices Again

Example

If a student repeatedly scores low in a topic:

- The system identifies the weak area.
- The Adaptive Agent selects easier learning material.
- Additional practice questions are provided.
- Feedback is given immediately.

Conclusion

The system uses **RDD-based large-scale analytics and intelligent agents** to provide personalized and adaptive learning experiences.

5. Reverse Engineering a Cloud-Based AI Monitoring System

Objective

To reconstruct a cloud system that automatically monitors resources, detects anomalies, and scales infrastructure in real time.

Inferred System Architecture

Servers + Applications + Network Logs

↓

Log Collection

↓

PySpark RDDs

↓

Distributed Log Processing

↓

Anomaly Detection

↓

Intelligent Agents

↙ ↓ ↘

Monitoring Agent | Scaling Agent | Response Agent

↘ ↓ ↙

Cloud Management

↓

Optimized Resources

1. Large-Scale Log Processing

The cloud platform collects:

- CPU usage
- Memory usage
- Network traffic

- Application logs
- Server errors
- Response time

PySpark RDDs distribute these logs across multiple machines for parallel processing.

2. RDD Operations

Possible operations include:

- map() – processes individual log records
- filter() – identifies error messages
- reduceByKey() – calculates resource usage
- groupByKey() – groups logs by server

3. Fault Tolerance

RDDs support fault tolerance through **lineage**.

If a worker node fails:

- The lost data can be recreated.
- Spark uses the sequence of transformations.
- Processing can continue without restarting the complete system.

4. Intelligent Agents

Monitoring Agent

- Continuously observes system performance.

Anomaly Detection Agent

- Detects unusual CPU, memory, or network behavior.

Scaling Agent

- Adds or removes computing resources based on demand.

Response Agent

- Responds to failures and system problems.

Example Decision Process

High CPU Usage Detected

↓

Monitoring Agent

↓

Anomaly Confirmed

↓

Scaling Agent

↓

Add More Cloud Resources

↓

System Performance Improves

Conclusion

The reverse-engineered cloud monitoring system uses PySpark RDDs for scalable processing, RDD lineage for fault tolerance, and intelligent agents for automated monitoring, scaling, and anomaly response.